



MULTI-AGENT AI ARCHITECTURE

Clinical Trial Matching Copilot

*Designing agent systems that know
when they are not sure.*

Live demo · LangGraph + Groq · Richard Abishai



The problem statement

Can one AI safely decide if a patient fits a clinical trial?

- Patient notes are messy free text.
- Trial rules are long and brittle.
- Missing data is extremely common.
- One model fills gaps with guesses.
- It then sounds confident and certain.
- We cannot see where judgment slipped.



A few terms, in plain words



Agent

An LLM given one job and rules.



Orchestration

Deciding which agent runs next.



Temperature

Creativity dial; zero means most predictable.



State

A shared whiteboard agents read/write.



JSON contract

A strict shape for each answer.



Inference

The model thinking and producing output.

The solution: three agents

*Split the one opaque step into
three honest, inspectable ones.*



Researcher

Extracts facts from the text.

Never decides eligibility.



Evaluator

Judges using only the evidence.

Allowed to say: not sure.



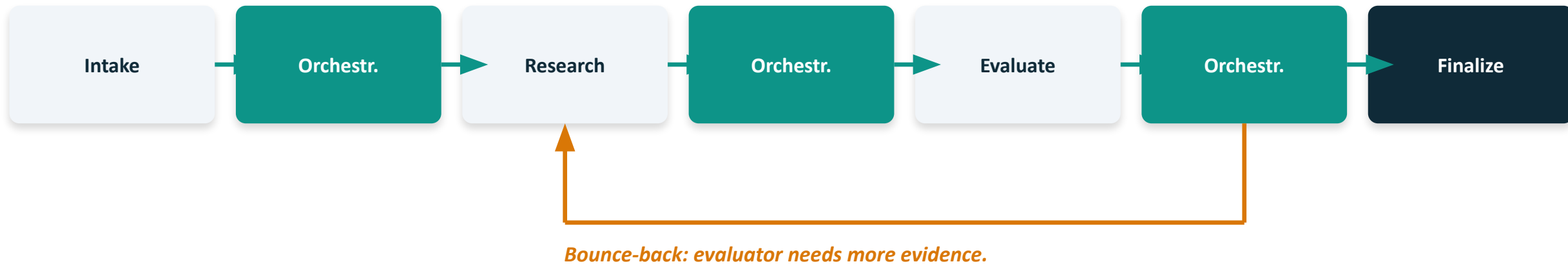
Orchestrator

Routes work between the agents.

Handles retries and finishing.

How the agents are wired

Shared state flows through each node.



- State plus nodes plus edges.
- Orchestrator sits between every step.
- Retry logic is explicit, not hidden.
- Every decision is visible and debuggable.



Code 1 of 4 — Shared State

The whiteboard every agent reads and writes.

```
class TrialState(TypedDict):  
    patient_case: str  
    trial_criteria: str  
    evidence_pack: dict  
    evidence_gaps: list  
    evaluator_report: dict  
    final_recommendation: str  
    next_step: str  
    retry_count: int  
    bounce_reason: str
```

What this does

- One typed object, shared by all.
- Each node returns a small update.
- “State in, partial state out.”
- `bounce_reason` carries the retry question.



Code 2 of 4 — The two specialists

Researcher extracts; Evaluator judges and may refuse.

```
# Researcher: extract, never decide
return {"evidence_pack": parsed,
        "evidence_gaps": gaps}

# Evaluator: judge from evidence
verdict in [
    "eligible",
    "unlikely_eligible",
    "unclear",
    "needs_more_evidence"]
```

What this does

- Researcher only structures the facts.
- It must list gaps, not hide them.
- Evaluator sees evidence, never raw notes.
- It can return needs_more_evidence.
- That is permission to be uncertain.



Code 3 of 4 — The orchestrator

The brain: it routes, and it handles the bounce-back.

```
# no evidence yet -> research
if not evidence_pack:
    next = "research"

# evaluator refused -> retry
if verdict == "needs_more_evidence"
    and retry_count < 1:
    next = "research"
    bounce_reason = question
    retry_count += 1
```

What this does

- Reads the state, picks the next step.
- If the evaluator says not enough,
- it sends work back to research.
- It attaches the evaluator's question.
- The retry counter prevents endless loops.



Code 4 of 4 — Build the graph

Add nodes, wire edges, compile before running.

```
builder = StateGraph(TrialState)
builder.add_node("research", ...)
builder.add_node("evaluate", ...)
builder.set_entry_point("intake")

builder.add_conditional_edges(
    "orchestrator", router, {...})

graph = builder.compile()
```

What this does

- Nodes are the agents we wrote.
- Conditional edges enable the routing.
- They make the bounce-back loop possible.
- `compile()` checks the wiring upfront.



Temperature, in one analogy

Low temperature (0)

- Same question, same answer.
- Careful, predictable, repeatable.
- Best choice for a live demo.

High temperature

- Same question, varied answers.
- Creative, surprising, less reliable.
- Good for brainstorming, not safety.

Think of it as a creativity dial.



Live: one agent corrects another



Researcher

First pass looks over-confident.



Evaluator

Asks: what are the liver values?



Orchestrator

Sends it back with that question.



Researcher

Second pass is honestly uncertain.



Finalize

Stops safely, flags the open question.



Remove the doubt, watch it fail

One line changed: `EVALUATOR_ENABLED = False`



Gatekeeper ON

- Flags the unresolved liver risk.
- Asks for the missing values.
- Refuses to over-claim eligibility.



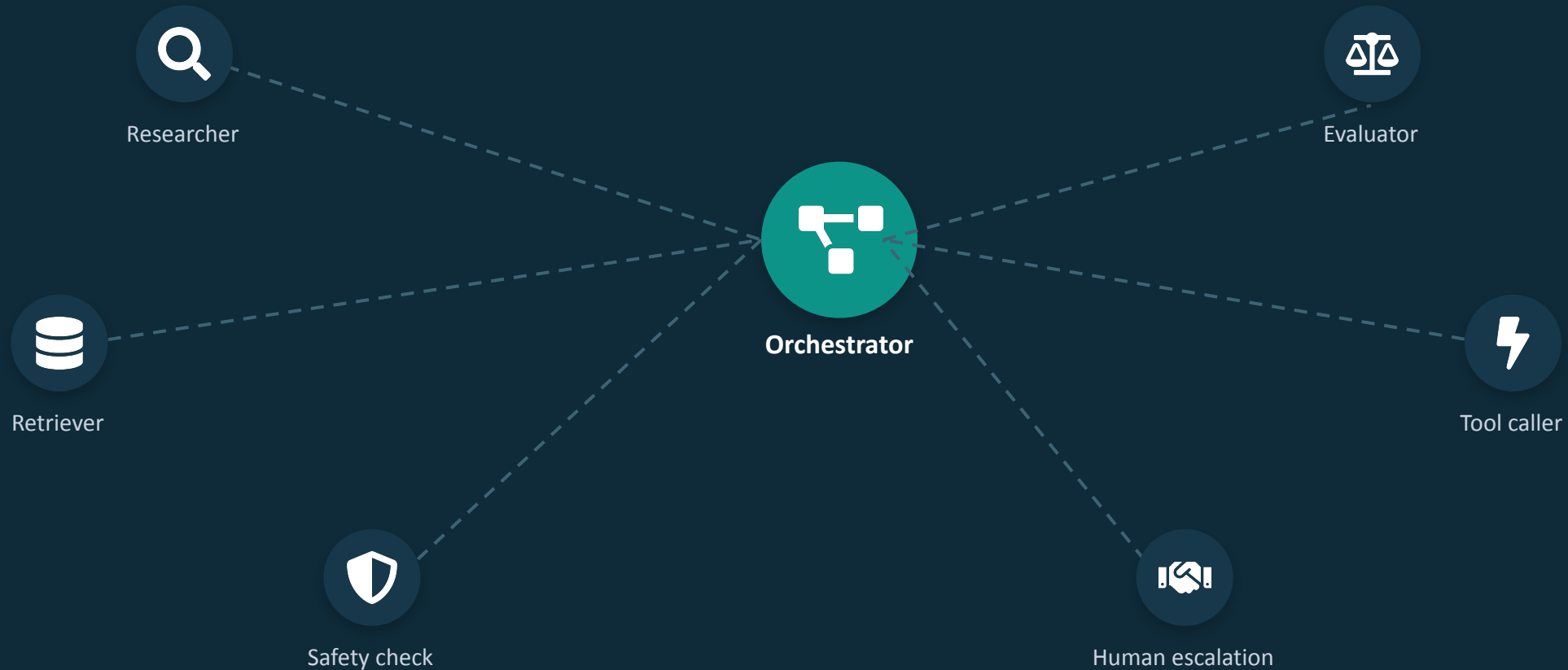
Gatekeeper OFF

- Declares the patient likely eligible.
- No caveat and no open question.
- Confidently wrong on real risk.

The architecture is the safety.

Where the orchestrator earns its keep

Our demo is small. Real systems are not — the brain gets busy.



It manages many agents, tools, retries, budgets and when to call a human.



Why Groq for this demo



Very fast inference

Low latency keeps a live demo snappy.



Simple integration

One package, one API key, done.



Open models

Runs Llama-3.3-70B out of the box.



Drop-in via LangChain

ChatGroq plugs straight into the graph.

The architecture is model-agnostic — swap Groq for any provider.

What this means for your work

- Separate gathering from deciding.
- Let agents admit being unsure.
- Make every retry visible and capped.



Fintech

Loan and KYC diligence.



Edtech

Student intervention triage.



R&D

Research paper screening.



Open to collaborate on research projects.
The code is yours — let's build together.



References & resources

LangGraph docs

docs.langchain.com/oss/python/langgraph

LangGraph StateGraph API

reference.langchain.com/python/langgraph

Multi-agent guidance

docs.langchain.com/oss/python/langchain/multi-agent

Groq + LangChain

console.groq.com/docs/langchain

Groq models & limits

console.groq.com/docs/models

Project repo

richardabish.ai